CineLog – Social Movie Diary Platform

Project Proposal Report & ER Diagram

Names	Roll numbers
Azaan Imran	**26100281**
Hanzalah Sohail Faraz	**26100028**
Muhammad Hashaam Tauseef	**26020105**
Raahem Azam	**26020347**

Description

CineLog is a proposed social movie diary platform (inspired by Letterboxd) that allows film enthusiasts to log movies they've watched, rate and review them, and share their cinematic journey with others. It serves as a personal film journal and a community hub in one; users can keep a diary of their film watching, create movie lists (e.g. favorite films or themed collections), and follow friends to see what they're watching or reviewing. The platform is built on a rich dataset of movies, people, and genres, so each film entry comes with detailed information (cast, crew, genre, etc.) to enhance discovery. Overall, CineLog aims to combine the utility of a movie database with the interactivity of a social network, providing an engaging way for movie fans to document and discuss films.

Problem Statement and Motivation

With the rise of streaming services and online content, movie fans lack a dedicated centralized space to track their watching habits and get personalized recommendations from friends. Many resort to generic sites or spreadsheets to log what they watch, and miss out on the social discovery features that dedicated film platforms offer. This gap is evidenced by the popularity of Letterboxd, often described as "Goodreads for movies", where members can record films, write reviews, and interact with fellow cinephiles. Letterboxd's user base exploded during the pandemic and now exceeds 20 million users (as of 2025), underscoring the strong demand for movie-centered social cataloging. Motivated by this, our project aims to create CineLog as an academic and practical endeavor. The goal is twofold: to solve the above problem by providing movie lovers (our target audience) with an interactive diary/review platform, and to give our

team experience in full-stack development and database design. By implementing CineLog, we intend to learn how to design a robust database (using Microsoft SQL Server) and develop a responsive web application (both back-end and front-end) that meets a real-world need. The end result will be a useful application for movie fans to document their film life and discover new movies through community engagement.

Key Objectives and Implementation Plan

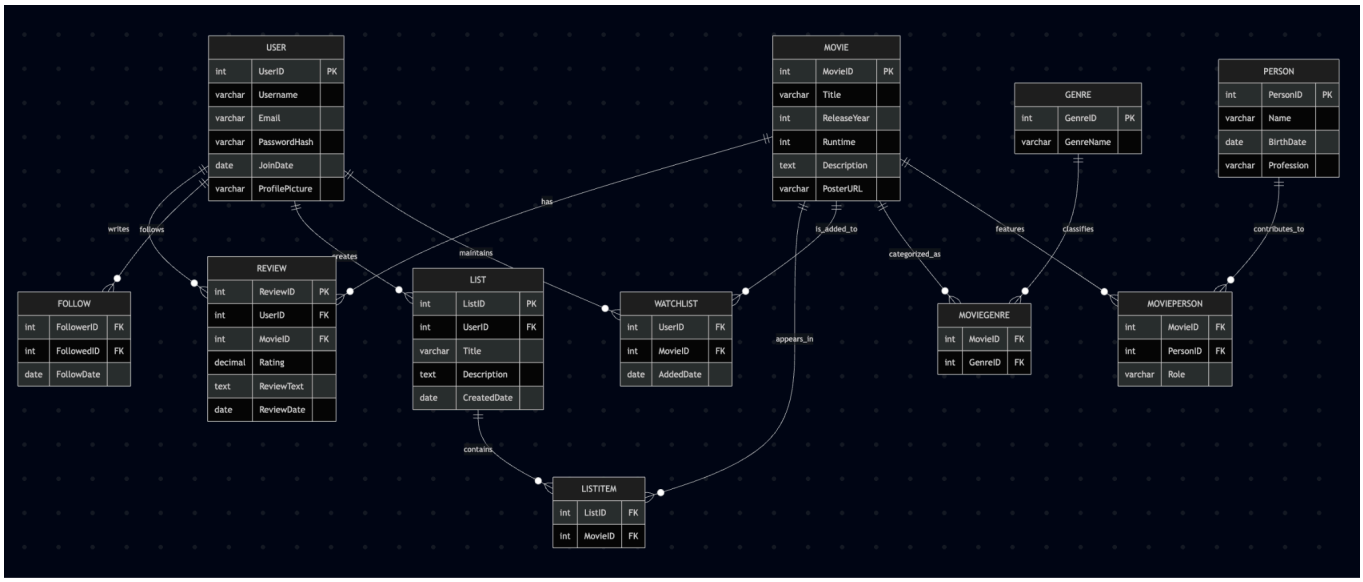
- **Comprehensive Movie Database**
  Goal: Store detailed movie, genre, and person data.
  Plan: Use relational tables (Movies, Genres, People) and join tables (e.g. MovieGenre, MoviePerson) to support rich metadata and search.
- **User Diary & Logging**
  Goal: Let users log movies they've watched with ratings/reviews.
  Plan: Create a Review table linking Users and Movies, with fields for date, rating, and review text.
- **Ratings & Reviews (Community Content)**
  Goal: Enable public movie reviews and star ratings.
  Plan: Display reviews on movie pages; store entries in the Review table with foreign keys for efficient queries and filtering.
- **Social Following & Activity Feed**
  Goal: Build social interaction through follows and activity updates.
  Plan: Create a Follow table; generate feed entries from user actions
- **Watchlists & Custom Lists**
  Goal: Let users save planned films and create themed movie lists.
  Plan: Use Watchlist, List, and ListItem tables. Enable adding/removing movies via front-end actions.
- **Full-Stack Integration**
  Goal: Build a responsive, user-friendly web app.
  Plan: Use Dotnet Framework for front-end, a backend API for logic, and MS SQL for database. Ensure performance with optimized queries and schema.

Main Features, Operations, and Outcomes

- **Personal Movie Diary**
  Operation: Users log each movie watched with a date, rating, and optional review.
  Outcome: Builds a chronological movie-watching history for personal reflection or tracking.
- **Ratings & Reviews**
  Operation: Users rate films (for example 1–5 stars) and write reviews visible to others.
  Outcome: Creates a public opinion hub, helping users discover and discuss films.

- **Watchlist (Planned Films)**
  Operation: Users add movies they want to watch to a personal list.
  Outcome: Helps users keep track of future viewings and recommendations.
- **Custom Lists (Curated Collections)**
  Operation: Users create themed lists (e.g., “Top 10 Comedies”) and add movies.
  Outcome: Enables sharing of personal taste and film recommendations.
- **Social Following**
  Operation: Users can follow other members on the platform to stay updated on their activity, such as the movies they log, rate, or review.
  Outcome: Encourages community interaction and movie discovery through social updates.

ER Diagram(made via mermaid code editor)



ER Diagram Relationship Types

One-to-One (1:1)

Currently, there are no one-to-one relationships in the diagram. All relationships involve either one-to-many or many-to-many connections.

One-to-Many (1:N)

- **User to Review:**
  One user can write many reviews. Each review references a single user.
- **User to List:**
  One user can create multiple custom lists.

- **User to Watchlist:**
  One user can add many movies to their watchlist.
- **Movie to Review:**
  One movie can have many reviews from different users.
- **List to ListItem:**
  One list can contain many movies.
- **Movie to ListItem:**
  One movie can appear in many different user-created lists.
- **Movie to MoviePerson:**
  One movie can have many associated people (actors, directors, etc.).
- **Person to MoviePerson:**
  One person can be involved in many movies.
- **Movie to MovieGenre:**
  One movie can belong to several genres.
- **Genre to MovieGenre:**
  One genre can apply to many different movies.

Many-to-Many (M:N)

- **Movie and Genre (via MovieGenre):**
  A movie can have multiple genres, and a genre can apply to multiple movies.
- **Movie and Person (via MoviePerson):**
  A movie can involve many people (cast/crew), and one person can work on many movies.
- **List and Movie (via ListItem):**
  A list can include many movies, and a movie can appear in many lists.
- **User and Movie (via Watchlist):**
  A user can watchlist many movies, and each movie can appear on many users' watchlists.
- **User and User (via Follow):**
  Users can follow many others, and be followed by many, forming a self-referential many-to-many relationship.